Assignment - 01

Course Title: Data Structure and Algorithms-II Lab
Course Code: CSE 2202

Prepared by:

Md. Ehasan Mahmud

Roll: 202122104026

Department: Computer Science and Engineering

Submitted to:

Dr. Sujan Chandra Roy

Lecturer

Department of Computer Science and Engineering

Bangabandhu Sheikh Mujibur Rahman University, Kishoreganj.

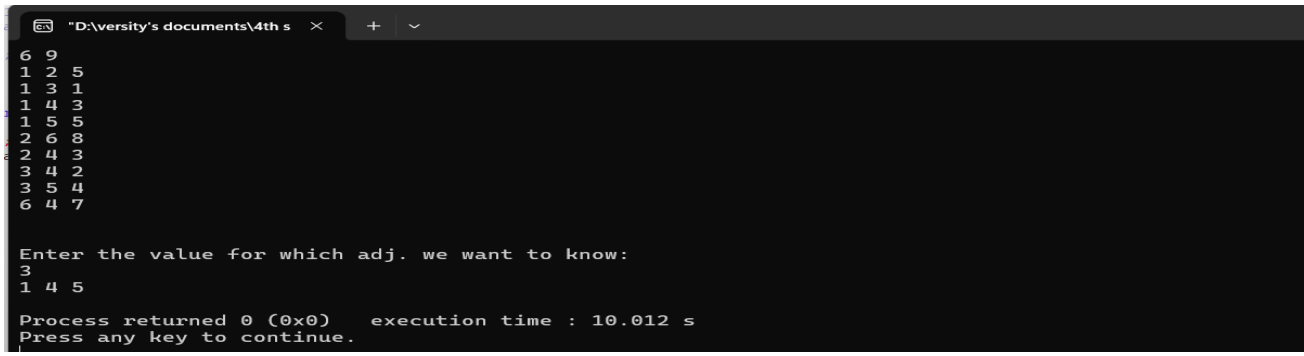
1.Implement the adjacency matrix of a given graph:

Code:

```
#include<bits/stdc++.h>
using namespace std;
vector<int>adj[100];
int main()
{
  int n,e;
  cin>>n>>e;
  int arr[n+1][n+1];
  for(int i=1;i<=n;i++){
   for(int j=1;j<=n;j++){
      arr[i][j]=0;
   }
  }
  while(e--){
   int u,v,w;
   cin>>u>>v>>w;
   adj[u].push_back(v);
   adj[v].push_back(u);
   arr[u][v]=w;
   arr[v][u]=w;
  }
  cout<<endl;
  /*cout<<"Adjacent Matrix are below: "<<endl;
  for(int i=1;i<=n;i++){
   for(int j=1;j<=n;j++){
      cout<<arr[i][j]<<" ";
   }
   cout<<endl;
  } */
  cout<<endl;
  int m;
  cout<<"Enter the value for which adj. we want to know: "<<endl;
  cin>>m;
  for(int i=0;i< adj[m].size();i++){
      cout<<adj[m][i]<<" ";
  }
  cout<<endl;


}
/*
6 9
1 2 5
1 3 1
1 4 3
1 5 5
2 6 8
2 4 3
3 4 2
3 5 4
6 4 7
*/
```

Output:

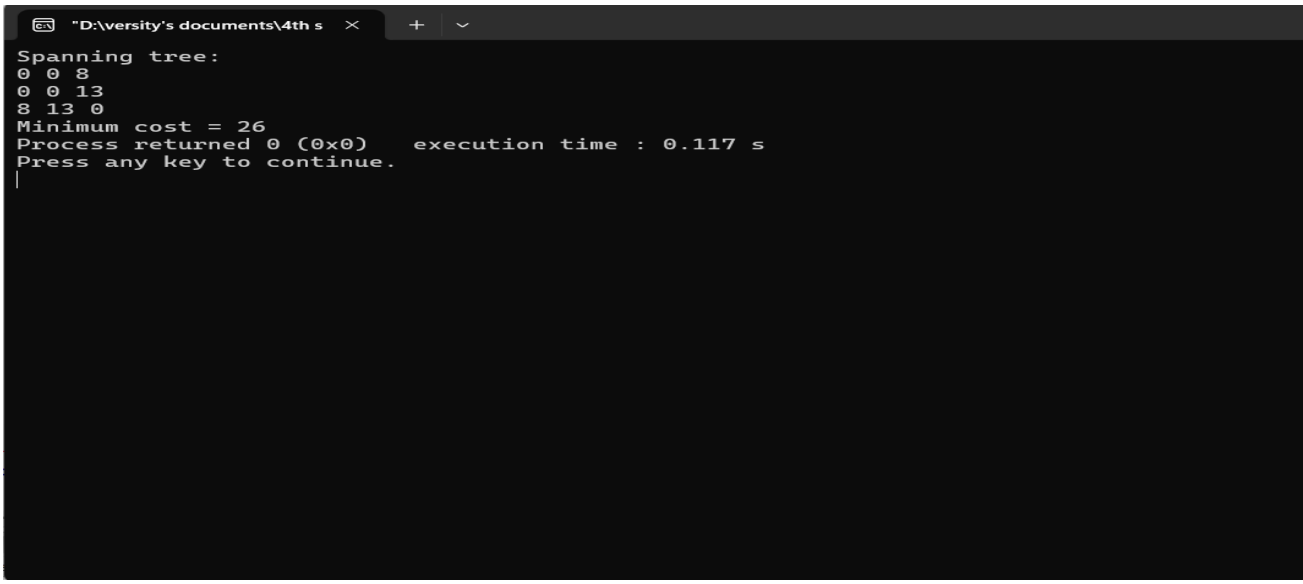


2. Prim's algorithm:

```
#include<bits/stdc++.h>
#define infinite 10e9
#define maxx 10
using namespace std;
int Graph[maxx][maxx]={
  {0, 19, 8},
  {21, 0, 13},
  {15, 18, 0}
};
int DD[maxx][maxx],n;
int prims();
int main(){
  int i,j,cost;
  n=3;
  cost=prims();
  cout <<"Minimum Spanning Tree:";
  for(i=0;i<n;i++) {
    cout<<endl;
    for(j=0;j<n;j++)
      cout<<DD[i][j]<<" ";
  }
  cout<<endl;
  cout<<" Minimum cost= "<<cost;
  return 0;
}
int prims(){
  int C[maxx][maxx];
  int u,v,min_d,dist[maxx],from[maxx];
  int visited[maxx],ne,i,min_c,j;
  for(i=0;i<n;i++)
    for(j=0;j<n;j++) {
      if(Graph[i][j]==0)
        C[i][j]=infinite;
      else
        C[i][j]=Graph[i][j];
```

```
      DD[i][j]=0;
    }
  dist[0]=0;
  visited[0]=1;
  for(i=1;i<n;i++) {
    dist[i]=C[0][i];
    from[i]=0;
    visited[i]=0;
  }
  min_c=0;
  ne=n-1;
  while(ne>0) {
    min_d=infinite;
    for(i=1;i<n;i++)
      if(visited[i]==0&&dist[i]< min_d) {
        v = i;
        min_d=dist[i];
      }
    u=from[v];
    DD[u][v]=dist[v];
    DD[v][u]=dist[v];
    ne--;
    visited[v]=1;
    for(i=1;i<n;i++)
      if(visited[i]==0 && C[i][v]<dist[i]) {
        dist[i]=C[i][v];
        from[i]=v;
      }
    min_c=min_c+C[u][v];
  }
  return(min_c);
}
```

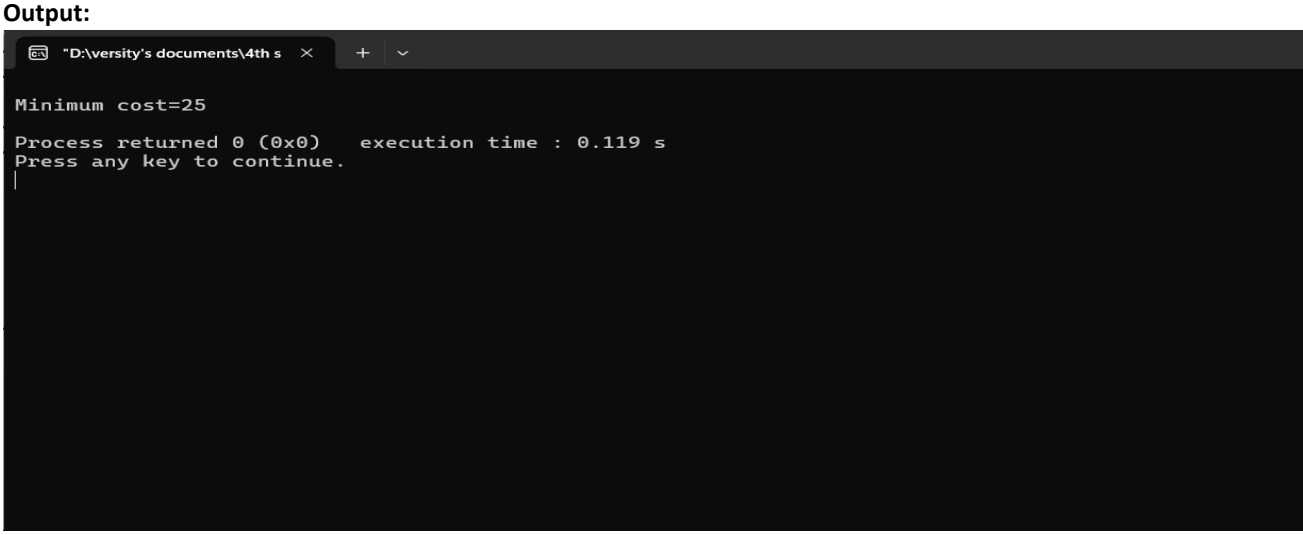
Output:



3. Kruskal's algorithm :

```
#include <bits/stdc++.h>
using namespace std;
const int inf=1e9;
int k,a,b,u,v,n,ne=1;
int mincost=0;
int cost[3][3]={
   {0, 10, 20},{12, 0, 15},{16, 18, 0}};
int p[9]={0};
int aFind(int i)
{
   while (p[i]!=0) {
      i=p[i];
   }
   return i;
}
int aUnion(int i,int j)
{
   if (i!=j) {
      p[j]=i;
      return 1;
   }
   return 0;
}
int main()
{
   n=3;
   for (int i=0;i<n;i++) {
      for (int j=0;j<n;j++) {
         if (cost[i][j]==0) {
            cost[i][j]=inf;
         }
      }
   }
   //cout <<"Minimum Spanning Tree:";
   cout<<endl;
   while (ne<n) {
      int min_val=inf;
      for (int i=0;i<n;i++) {
         for (int j=0;
               j<n;j++) {
            if (cost[i][j]<min_val) {
               min_val=cost[i][j];
               a=u=i;
               b=v=j;
            }
         }
      }
      u=aFind(u);
      v=aFind(v);
      if (aUnion(u, v)!=0) {
         //cout <<a<<"->"<<b<<"\n";
         mincost +=min_val;
      }
      cost[a][b]=cost[b][a]=999;
      ne++;
   }
   cout <<"Minimum cost="<<mincost<<endl;
   return 0;
}
```

Output:



4. Bellman-Ford:

```
#include<bits/stdc++.h>
using namespace std;
    struct Edge
    {
       int source,dest,weight;
    };
    void Path(vector<int> const &parent,int v)
    {
       if (v<0)
          return;
       Path(parent,parent[v]);
       cout <<v<<" ";
    }
    void Bell_f(vector<Edge> const &edges,int source,int N)
    {
       int E=edges.size();
       vector<int>distance(N, INT_MAX);
       distance[source]=0;
       vector<int>parent(N,-1);
```

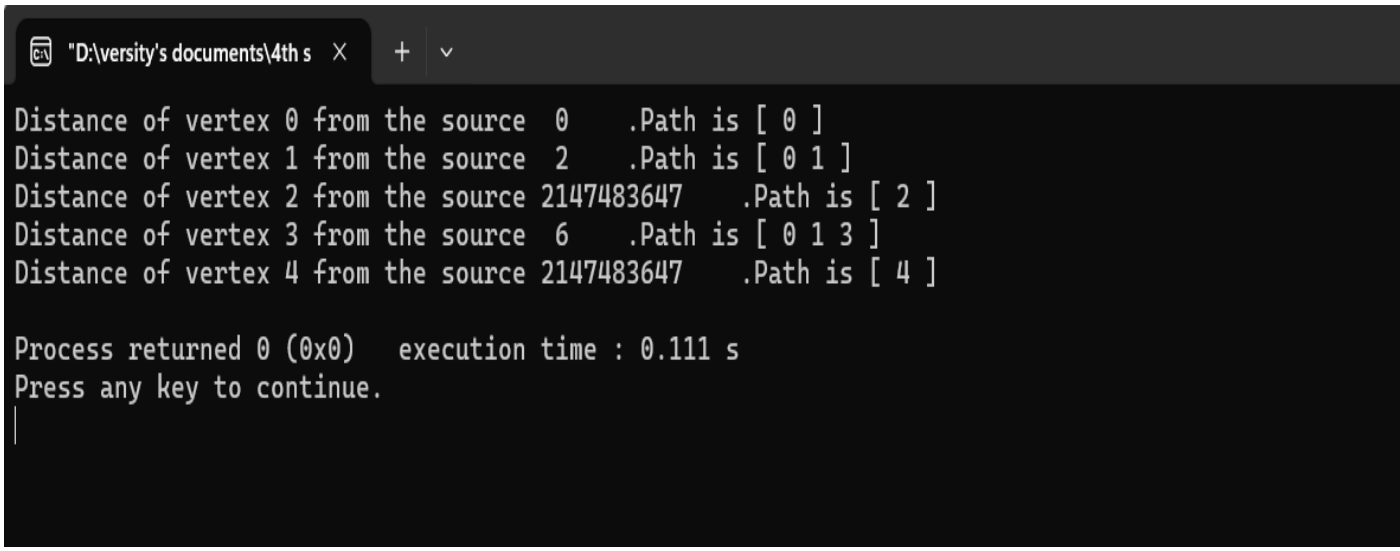
```
    int u,v,w,k=N;
    while(--k)
    {
        for(int j=0;j<E;j++)
        {
            u=edges[j].source,v=edges[j].dest;
            w=edges[j].weight;

            if(distance[u]!=INT_MAX && distance[u]+w<distance[v])
            {
                distance[v]=distance[u]+w;
                parent[v]=u;
            }
        }
    }
    for (int i=0;i<E;i++)
    {
        u=edges[i].source,v=edges[i].dest;
        w=edges[i].weight;

        if(distance[u]!=INT_MAX && distance[u]+w<distance[v])
        {
            cout << "Negative Cycle Here.";
            return;
        }
    }
    for (int i=0;i<N;i++)
    {
        cout<<"Distance of vertex "<<i<<" from the source "
            <<setw(2)<<distance[i]<<"    .Path is [ ";
        Path(parent,i);cout<<"]"<<'\n';
    }
}
```

```
int main()
{
   vector<Edge>edges=
   {
      { 0, 1, 2 },{ 1, 3, 4 },{ 0, 3, 6 }
   };
   int N=5;
   int source=0;
   Bell_f(edges,source,N);
   return 0;
}
```

output:

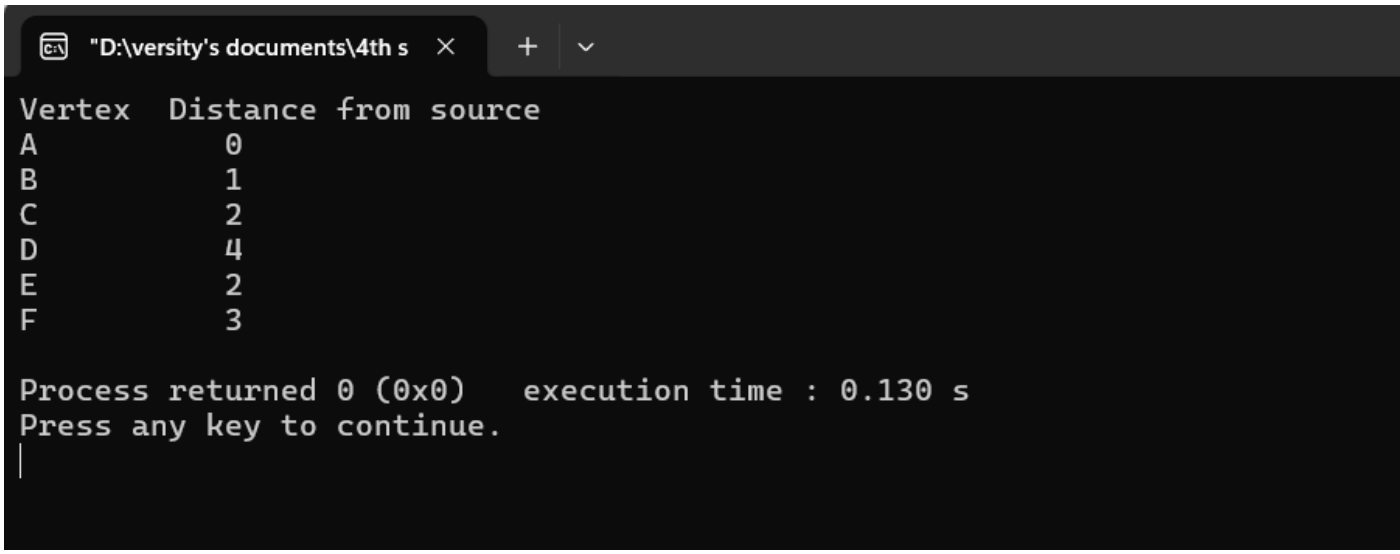


5. Dijkstra's algorithm:

```
#include<bits/stdc++.h>
using namespace std;
        int min_dist(int Dis[], bool Vis[]){
          int Min=INT_MAX,id;
          for(int k=0; k<6; k++) {
            if(Vis[k]==false && Dis[k]<=Min) {
              Min=Dis[k];
              id=k;
            }
          }
          return id;
        }
        void G_dij(int graph[6][6],int src){
          int dist[6];
          bool visited[6];
          for(int k = 0; k<6; k++) {
            dist[k] = INT_MAX;
            visited[k] = false;
          }
          dist[src] = 0;
          for(int k = 0; k<6; k++) {
            int m=min_dist(dist,visited);
            visited[m]=true;
            for(int k = 0; k<6; k++) {
              if(!visited[k] && graph[m][k] && dist[m]!=INT_MAX && dist[m]+graph[m][k]<dist[k])
                dist[k]=dist[m]+graph[m][k];
            }
          }
          cout<<"Vertex  Distance from source"<<endl;
          for(int k = 0; k<6; k++) {
            char St=65+k;
            cout<<St<<"         "<<dist[k]<<endl;
```

```
    }
  }
  int main(){
    int graph[6][6]={
      {0, 1, 2, 0, 0, 0},
      {1, 0, 0, 5, 1, 0},
      {2, 0, 0, 2, 3, 0},
      {0, 5, 2, 0, 2, 2},
      {0, 1, 3, 2, 0, 1},
      {0, 0, 0, 2, 1, 0}
    };
    G_dij(graph,0);
    return 0;
  }
```

output:

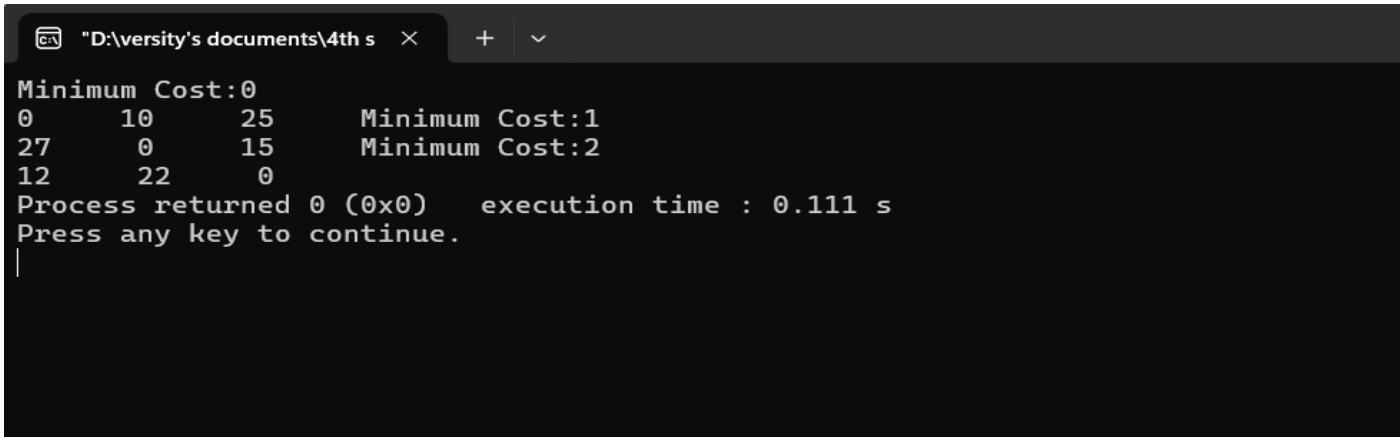


6. Floyd-Warshall algorithm:

```
#include <bits/stdc++.h>
using namespace std;
void FloydW(int b[][3]){
   int i,j,k;
  for (int k=0;k<3;k++) {
     for (int i=0;i<3;i++) {
        for (int j =0;j<3;j++) {
           if ((b[i][k]*b[k][j]!=0) && (i!=j)) {
              if ((b[i][k]+b[k][j]<b[i][j]) || (b[i][j]==0)) {
                 b[i][j]=b[i][k]+b[k][j];
              }
           }
        }
     }
  }
  for (i=0;i<3;i++) {
     cout<<"Minimum Cost:"<<i<<endl;
     for (j=0;j<3;j++) {
        cout<<b[i][j]<<"     ";
     }
  }
}
int main(){
  int b[3][3];
  for (int i=0;i<3;i++) {
     for (int j=0;j<3;j++) {
        b[i][j]=0;
     }
  }
  b[0][1]=10;
```

```
  b[1][2]=15;
  b[2][0]=12;
  FloydW(b);
  return 0;
}
```

output:



7. Johnson's algorithm :

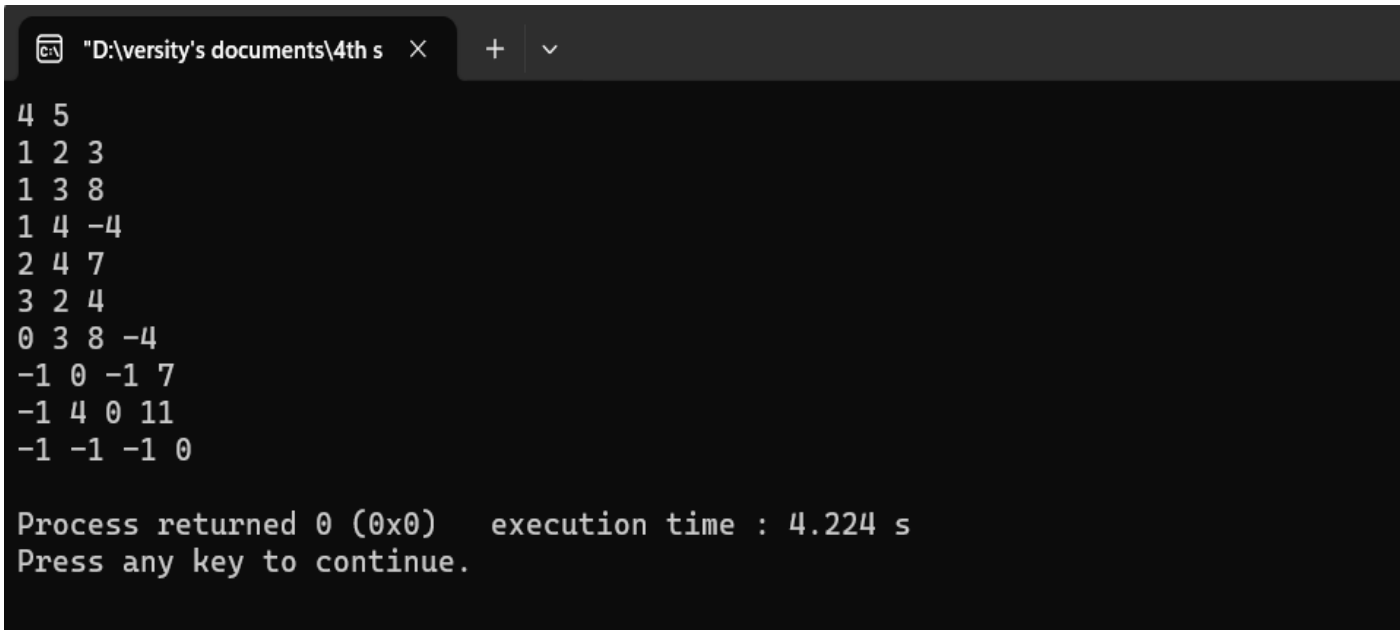
```
#include<bits/stdc++.h>
#define int long long
using namespace std;
const int INF=1e18;
const int NN=500;
vector<pair<int,int>>G[NN];
vector<int>Bell_f(int N,vector<vector<int>>&edges){
   vector<int>r(N+1,INF);
   r[0]=0;
   for(int i=0;i<=N;i++){
      bool check=false;
      for(auto&e:edges){
         int u=e[0],v=e[1],w=e[2];
```

```
            if(r[u]!=INF&&r[v]>r[u]+w){
                r[v]=r[u]+w;
                check=true;
            }
        }
        if(!check)break;
    }
    return r;
}
void Dijk(int src,vector<vector<int>>&path,int N){
    priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>pq;
    vector<int>dist(N+1,INF);
    dist[src]=0;
    pq.push({0,src});
    while(!pq.empty()){
        int d=pq.top().first,u=pq.top().second;
        pq.pop();
        if(d>dist[u])continue;
        for(auto&[v,w]:G[u]){
            if(dist[u]+w<dist[v]){
                dist[v]=dist[u]+w;
                pq.push({dist[v],v});
            }
        }
    }
    path[src]=dist;
}
signed main(){
    int N,M;
    cin>>N>>M;
    vector<vector<int>>edges;
```

```
for(int i=0;i<M;i++){
    int u,v,w;
    cin>>u>>v>>w;
    edges.push_back({u,v,w});
}
for(int i=1;i<=N;i++){
    edges.push_back({0,i,0});
}
vector<int>C=Bell_f(N,edges);
for(auto&e:edges){
    e[2]+=(C[e[0]]-C[e[1]]);
}
for(auto&e:edges){
    if(e[0]!=0){
        G[e[0]].push_back({e[1],e[2]});
    }
}
vector<vector<int>>path(N+1,vector<int>(N+1,INF));
for(int i=1;i<=N;i++){
    Dijk(i,path,N);
}
for(int i=1;i<=N;i++){
    for(int j=1;j<=N;j++){
        if(path[i][j]==INF){
            cout<<-1<<" ";
        }else{
            cout<<path[i][j]-(C[i]-C[j])<<" ";
        }
    }
    cout<<endl;
}
```

```
    return 0;
}
/*
4 5
1 2 3
1 3 8
1 4 -4
2 4 7
3 2 4
*/
```

Output:

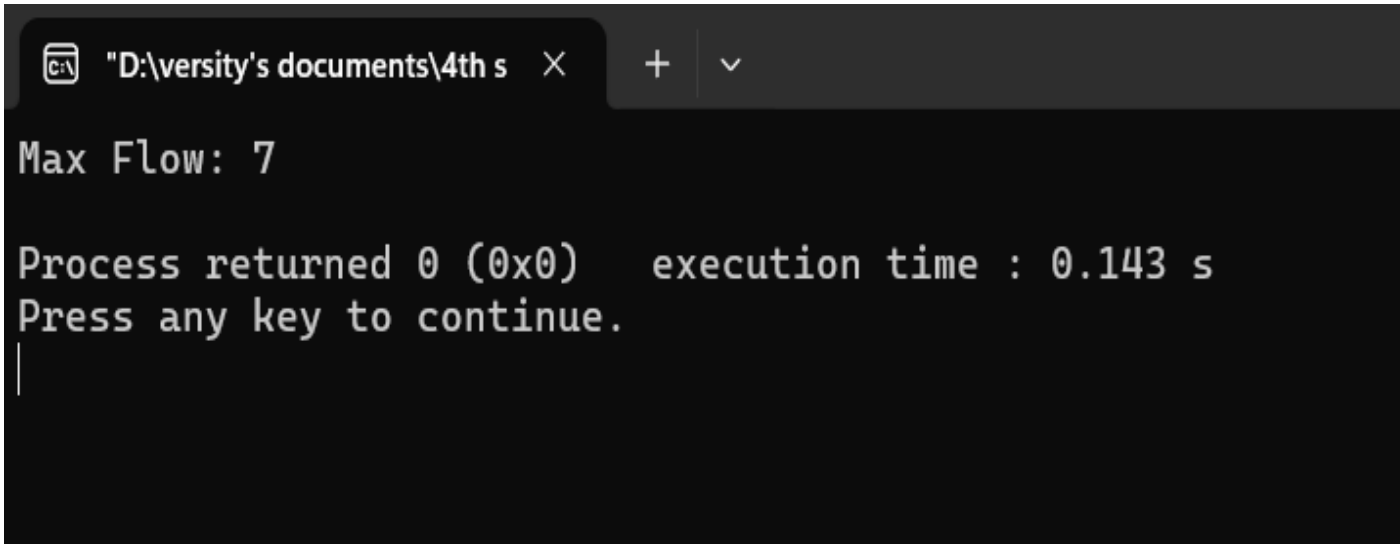


8. Ford-Fulkerson algorithm:

```
#include <bits/stdc++.h>
using namespace std;
int BFS(int Src,int target,int n,vector<int>& parent,vector<vector<int>>& graph){
    fill(parent.begin(),parent.end(),-1);
    parent[Src] = -2;
    queue<pair<int,int>> q;
    q.push({Src,1e9});
    while(!q.empty()){
        int u = q.front().first, cap = q.front().second;
        q.pop();
        for(int v=0;v<n;v++){
            if(u!=v && graph[u][v]!=0 && parent[v]==-1){
                parent[v] = u;
                int min_cap = min(cap,graph[u][v]);
                if(v==target)return min_cap;
                q.push({v,min_cap});
            }
        }
    }
    return 0;
}
int f_Fulker(int source,int target,int n,vector<vector<int>>& graph){
    vector<int> parent(n,-1);
    int max_flow = 0,min_cap = 0;
    while(min_cap = BFS(source,target,n,parent,graph)){
        max_flow += min_cap;
        int v = target;
        while(v!=source){
```

```
            int u = parent[v];
            graph[u][v] -= min_cap;
            graph[v][u] += min_cap;
            v=u;
        }
    }
    return max_flow;
}
void addEdge(vector<vector<int>>& graph,int u,int v,int w){
    graph[u][v] = w;
}
int main(){
    int V = 6;
    vector<vector<int>> graph(V,vector<int>(V,0));
    addEdge(graph,0,1,4);
    addEdge(graph,0,3,3);
    addEdge(graph,1,2,4);
    addEdge(graph,2,3,3);
    addEdge(graph,2,5,2);
    addEdge(graph,3,4,6);
    addEdge(graph,4,5,6);
    cout << "Max Flow: " << f_Fulker(0,5,V,graph) << endl;
    return 0;
}
```

output:


"D:\versity's documents\4th s
Max Flow: 7
Process returned 0 (0x0)   execution time : 0.143 s
Press any key to continue.